Vișănescu Dragoș Nicholas Babeș-Bolyai University 20.10.2026

Double Slit Experiment Simulator

Guide

This document serves as a guide to building a simulator for the double slit experiment, based on a project I developed myself. In this file you will find a step by step tutorial and breakdown of everything one would need to correctly implement this concept and genuinely understand the mathematics and physics behind it.

My personal implementation uses C++ for the actual computing-heavy simulation and a short Python script for plotting the results in a representative animation. In no way is this project bound to any fixed programming language choices, so feel free to use the ones you are most comfortable with.

As for prerequisites, this project surprisingly does not require any advanced mathematics or physics (as far as understanding what we actually code and what everything largely means), though for a deep and complete understanding of the physical phenomenon and the mathematical reasoning down to the roots, a deeper knowledge would certainly be required. Thus, one should be comfortable with the following concepts to proceed smoothly:

1. Mathematical knowledge prerequisites
- Complex Numbers (binary operations, absolute value, Euler's formula)
- Simple Real Functions
- Derivatives

2. Programming knowledge prerequisites
- Floating-point arithmetic
- Matrices
- Functions
- (Optional) OOP

3. Physics (basic understanding)
- Subatomic Particles (Electrons, Photons, etc.)

With these things in mind, we will be touching on more advanced subjects, but as previously mentioned, just understanding what everything does is not as daunting as it looks. Some such subjects include: multivariable functions, partial derivatives, the Schrödinger Equation and more. This paper is especially focused on explaining these concepts to people who are only expected to have experience with the prerequisites listed above, and to show that even something that seems this complicated and frightening can actually become surprisingly easy and intuitive.

Contents

I. The Double Slit Experiment 3

II. Physics and Mathematics Behind the Experiment and Simulation 6
- The Schrödinger Equation
- The Wave Function
- Partial Derivative of the Wave Function with Respect to Time
- Potential Energy
- Laplacian Operator

III. The Target 11
- What Is a Simulation, Actually?
- What Do We Even Compute?

IV. The Digital Representation of the System 14

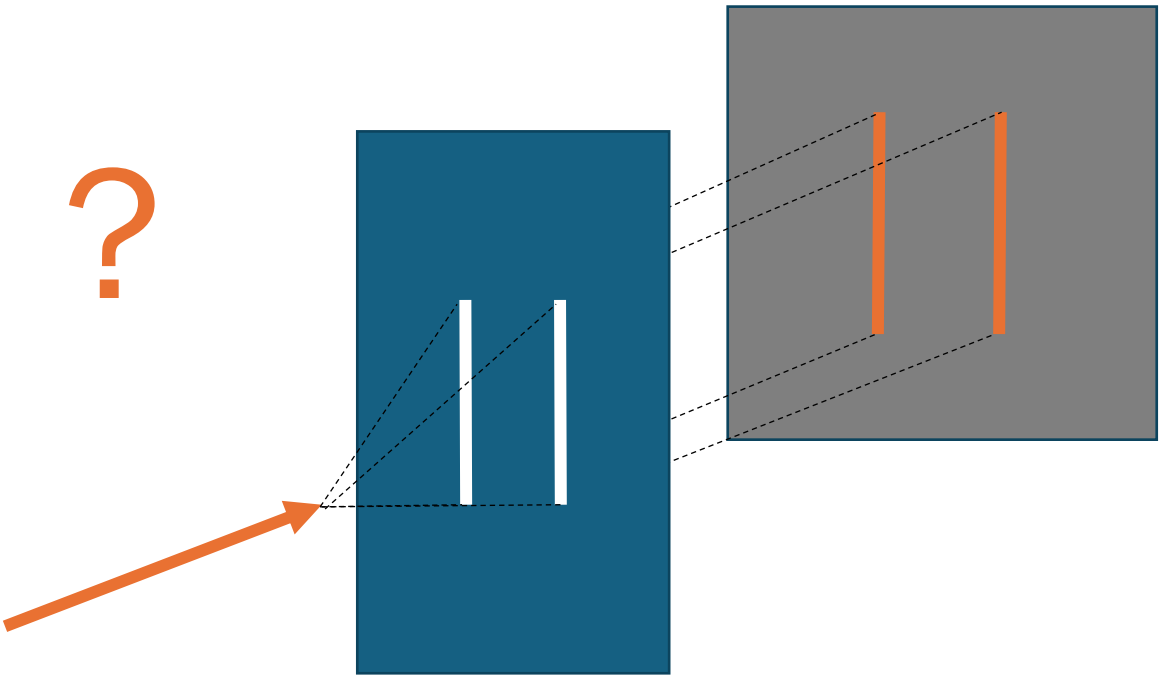
V. The Implementation 16
- 1. Implementation of the Spatial Representation
- 2. Implementation of the Particle Representation
- 3. Evolving the System
- 4. Upgrade

VI. Conclusions 24

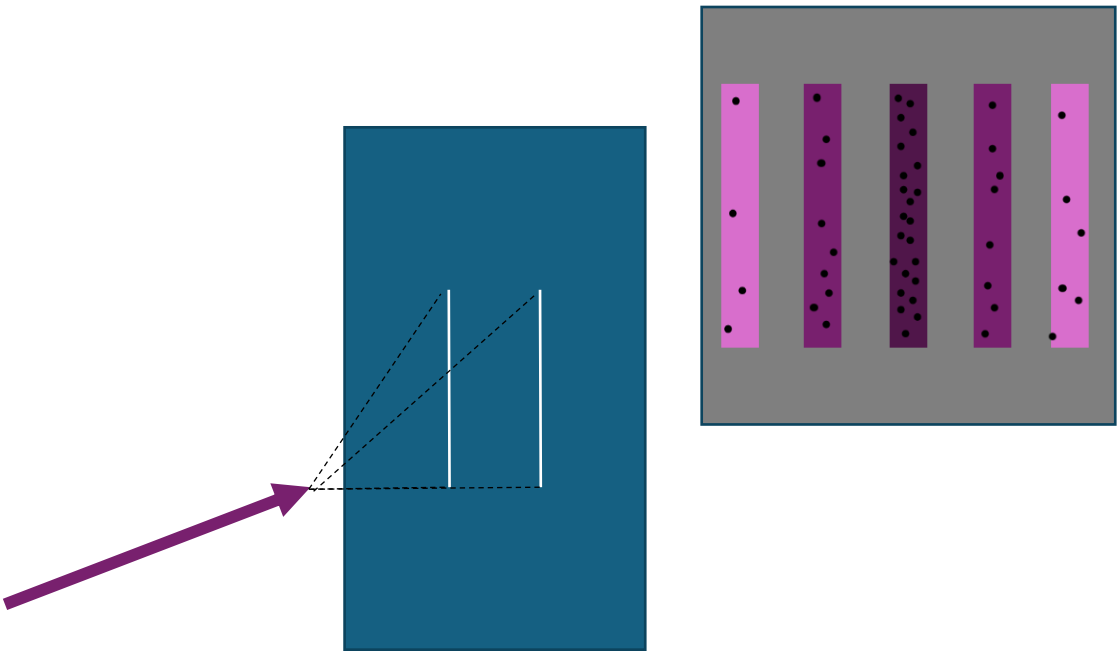
I. The Double Slit Experiment

Before we get to implementing the simulator we must first understand what we actually want to simulate. For this I will begin by briefly explaining the actual experiment and what it represents (you may skip this first chapter if you already know what the experiment does and how it is carried out).

First implemented in 1801 by Thomas Young, the double slit experiment is a way of showing that all known particles can actually exhibit wave-like behaviour. Young first showed that light interferes, while the same pattern was observed later on in multiple particles. Consider the following setting: we place a wall with two narrow slits standing upright, with a panel behind it. From the other side we shine a light right at the slits (as in the picture below). What might you expect to see on the back panel?



At first glance it might be very obvious that you would see a "projection" of the two slits on the back panel, since the wall blocks light, or any other kind of object, from passing through, except through the two slits.
In most cases, this assumption is actually perfectly correct. The only nuance is that, once we make the slits narrow enough and shoot the right kind of thing through them (maybe something very small like a photon or an electron), something very interesting happens.

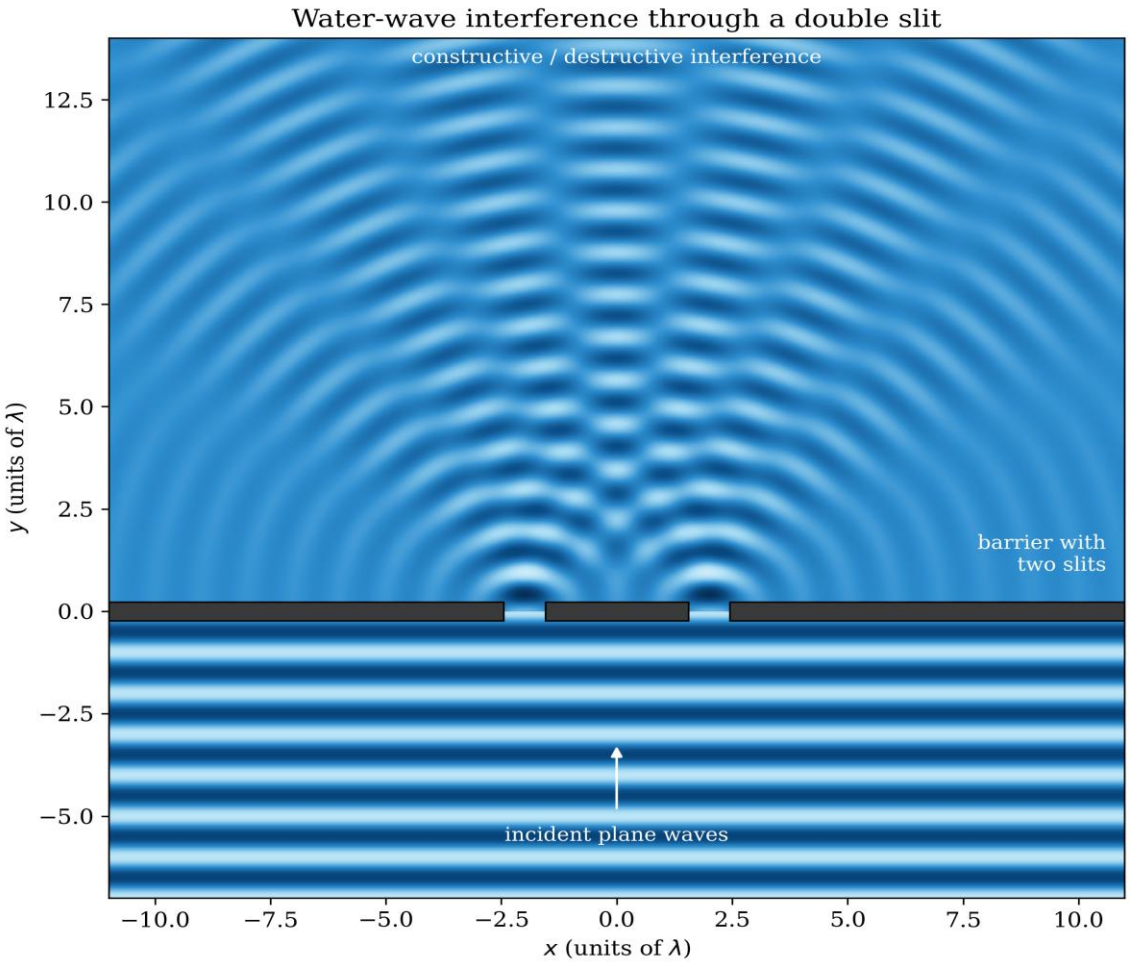


The actual projection after shooting a lot of particles through the slits is a collection of fringes in a "zebra-like" pattern! This unexpected result is what we call an interference pattern and is actually what we do this whole experiment for. This is the result that demonstrates a particle behaves like a wave.
Now the reasonable question is: how does this result imply a wave-like behaviour? How does this tell us that an electron or a photon is actually a wave in a way?

A very simple way of checking that this behaviour is specific to waves is by observing the most common and simple waves we can find in nature: moving water waves. If we repeated this same procedure, only creating waves in the water instead of shooting particles, we would get a surprisingly similar pattern in the end, only in a different context.

Looking at such a phenomenon, we can observe both the constructive and destructive interference, which are the exact actors that form the fringes: destructive interference allows for empty, untouched zones, while constructive interference hits the zones in between, thus alternating them.

The exact same thing happens with particles, only in order to actually observe it we must shoot multiple instances one after another so that the pattern can actually form, as the interference pattern in their case represents something a little different, of which we will talk more later on. More precisely, this wave behaviour of particles is described by the wave function, which has the additional functionality of providing a crucial insight: the probability of a particle being present at a certain point in space, at a moment in time.



With this short introduction to the experiment, how it works and what it shows us, we can move on to the actual physics and maths behind it. This is the core part of both the experiment and the simulation, so take your time and try to understand as much of what follows as possible. We won't describe the experiment any further, since there are already numerous sources on this. A last important addition would be that this experiment has several variations which might look alike, but imply very different phenomena in some cases. Some such differences include: photon vs electron, water waves, detection of path. Each of these variants touches on a slightly different topic and behave quite differently in practice. For our simulator, we will use an electron as the shooting particle and won't be using any path detection method.

II. Physics and Mathematics Behind the Experiment and Simulation

The central piece for everything that follows regarding both the mathematical understanding of the phenomenon portrayed by the experiment and the actual simulator development is without a doubt The Schrödinger Equation.
An important step we must take before we actually look at the equation is understanding what it is and what it does. The best way to find out the answer to both of these questions for any abstract and particularly difficult concept is to split it into subproblems, kind of like a Divide et Impera approach. So let's start from the Schrödinger Equation and work ourselves down to concepts we are already familiar with:

The Schrödinger Equation = "A *partial differential equation* governing the *wave function* of a *non-relativistic quantum-mechanical system*."

Now, seeing this definition, any normal person would probably mentally squirm trying to make sense of it. So let's continue with our Divide et Impera approach and tackle this definition by further splitting it into concepts that get closer and closer to our comfort zone. I underlined the seemingly abstract ideas in this definition and now we are going to list them one by one, repeating the process.

Partial Differential Equation = "An equation involving a *multivariable function* and one or more of its *partial derivatives*."

Multivariable function = "A function with more than one argument."

Partial derivative = "A derivative of a *multivariable function* with respect to one of its variables, while the other variables are treated as constants."

You might be surprised to find out we just got to a place where we can completely understand a whole branch of the initially terrifying complex concept! What we just did is we took the *partial differential equation* part of the definition and boiled it down to things we are completely familiar with: simple functions and derivatives!

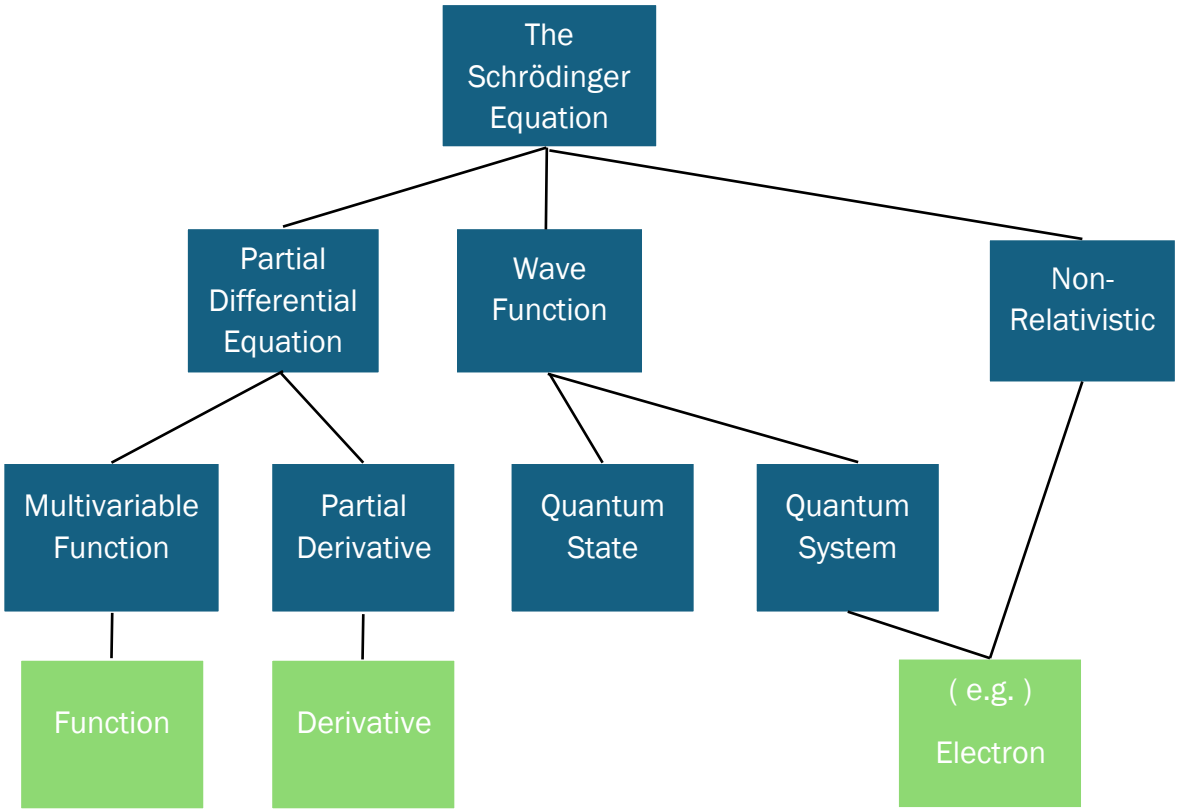
Wave Function = "The mathematical function used to represent the *quantum state* of a *quantum system*."

Quantum System = "The thing(s) we study or apply quantum mechanics to (e.g. an electron, a photon, an atom, a molecule, etc.)"

Quantum State = "The description of a *quantum system's* current state in quantum terms (e.g. the wave function is a representation of the *quantum state* of a *quantum system*), usually using probabilities, as concrete values are usually not known in quantum mechanics"

Non-Relativistic = "A concept not involving Albert Einstein's Theory of Relativity"

Believe it or not, we just stripped The Schrödinger Equation down to a few definitions we can now make sense of! Below you can observe a diagram of the breakdown we just did.



With this scheme in mind, I highly recommend you read as much as you can about each of these pieces and be comfortable with each level of the breakdown. You don't need to be able to write a paper on each of them, but you should be capable of explaining to someone what each of those notions is; that is a good sign you have grasped the concepts.

At this point you should have a clear definition in mind of The Schrödinger Equation, so let us continue with the actual mathematical form of it.

More precisely, we will be analysing *the time-dependent Schrödinger equation*, as we are interested in simulating a process and monitoring its evolution over time.

Time-dependent Schrödinger Equation:

$$i\hbar\frac{\partial\Psi}{\partial t} = \left(-\frac{\hbar^2}{2m}\nabla^2 + V\right)\Psi$$

Ψ = the wave function
i = the imaginary Unit
$\hbar$ = the reduced Planck constant
$\frac{\partial\Psi}{\partial t}$ = partial derivative of the wave function (Ψ) with respect to time
m = mass of the particle
V = potential energy of the particle
∇^2 = the Laplacian operator

By this description of the equation we can see what each term is and what it represents. The problem is that we don't really know what each of them does or how they even help us. So let's take each term separately and talk a bit about them.

1. Ψ - The Wave Function

   Probably the most important part of the Schrödinger Equation, the wave function, as we saw above when we broke down the definitions, is a representation of the quantum state of the particle we are analysing. Right now, though, we will focus more on the actual mathematical structure of it and how it helps us towards our goal. The wave function has two sets of arguments: the time and the positional coordinates. For the sake of our simulation, we will work in a 2D space, thus our wave function will look something like $\Psi(x, y, t)$, with the pair (x, y) denoting the spatial coordinates and t the time. This aspect of the function surprisingly doesn't concern or help us that much, as we will see in a minute, but we should be aware of what values this function actually gets as input.

   The most important part of the wave function is its output, because this value is what our whole computation will work towards obtaining. The wave function outputs a complex number called the probability amplitude. While this complex number carries a lot of information, we are only interested in one use of it, namely computing the probability density $|\Psi|^2$. Summing up this probability density over an area will ultimately give us the probability of finding our particle in that space at the time moment t .

2. $\frac{\partial \Psi}{\partial t}$ - Partial Derivative of the Wave Function with respect to time

   Directly tied to the wave function itself, this partial derivative lets us inspect the evolution of the wave function over time, exactly what we intend to do in our simulator. This piece is crucial in allowing us to approximate the wave function's value at a further moment in time, using only information we have at a present moment. By slightly altering the Schrödinger Equation, we can actually obtain a direct relation between this partial derivative and the wave function itself; this will become an extremely important tool for us.

$$\frac{\partial \Psi}{\partial t} = \left(\frac{i\hbar}{2m}\nabla^2 - \frac{iV}{h}\right)\Psi$$

   What this form of the equation does is allow us to directly calculate the partial derivative by only knowing the value of the function and some additional common terms.

3. V - Potential Energy

   Simply the potential energy of the particle at a certain point (x, y). While this can also vary with time, we will, for simplicity, consider it constant. V is thus also technically a function, but we will not treat it that way in our simulation. So, for us, V will simply be a set of known potential energy values at all the points in our plane.

4. ∇^2 - Laplacian Operator

   While this may look like another obscure part of the equation, its definition is actually pretty straightforward: the Laplacian Operator tells us how sharply the wave function bends. This is achieved by the simple calculation:

$$\nabla^2 = \frac{\partial \Psi^2}{\partial x^2} + \frac{\partial \Psi^2}{\partial y^2}$$

   As we can see, the operator is just the sum of the second spatial partial derivatives of the function. Fortunately, in practice we will be able to approximate this value with pretty high accuracy without too much hassle, as we will see further on, so this should not bother you in any way.

As for the rest of the terms in the equation (namely), we will not get into much detail as it is simply redundant for our purpose. In practice we will even treat these as non-dimensional constants by assigning simple values like 1 to most of them (except), as they bring no value to the actual calculation and working in S.I. units is not mandatory. Their actual purpose in the equation is tied to the physical explanation of the relation which we will not get into as it can get pretty complicated and exceeds our scope.

Before we conclude our mathematical analysis of the problem, we need to touch on a few important points that arise from what we just laid out about the Schrödinger Equation.

- First of all, we have to observe that the equation simply does not work for relativistic particles. Even though this is specified in its definition, it is important to point this out as this means our current process cannot be applied to particles like photons that have no mass. This is why for this simulator we will assume the particle in use is an electron.

- Second of all, if you have studied the Schrödinger Equation before or decide to look it up, you might find it in slightly different forms. Make sure you are checking the time-dependent equation, as this is what we use for our current project. Also note that the compact form $\mathbf{i\hbar\, \partial\Psi/\partial t = \hat{H}\Psi}$ is equivalent to what we used above, where $\hat{H}$ is simply the Hamiltonian operator, equivalent to our bracketed term $\mathbf{[\, -(\hbar^2/2m)\nabla^2 + V\,]}$.

- Looking at the equation from afar, it actually consists of three main parts: the term $\mathbf{-(\hbar^2/2m)\nabla^2}$ is the kinetic energy; the term $\mathbf{V}$ is the potential energy; and the sum of the two, $\mathbf{\hat{H} = -(\hbar^2/2m)\nabla^2 + V}$ (the Hamiltonian), is exactly the total energy. The left side of the equation, $\mathbf{i\hbar\, \partial/\partial t}$, is also the total energy, only seen from the time side.

III. The Target

By this point you should've passed through at least 8 pages of physics and mathematics, if not more, if you've done your own research on the subject, but you are still probably confused by one question: "what is our goal here?", and the truth is, we haven't answered that yet.

Now an obvious answer is "to simulate the double slit experiment", but again, how do we do that? How does everything we went through up until now help us in any way and how do we even achieve this? This chapter is designed to answer these exact questions and by the end of this you should know exactly what we are writing our code for and how we plan on simulating the experiment.

What is a simulation actually?

In our context, a simulation is an actual unfolding of events carried out over a period of time. More precisely, we want to carry out the course of events of the double slit experiment in a digital environment (our program).

So what does this imply? Well, let's analyse what roughly happens in the experiment:

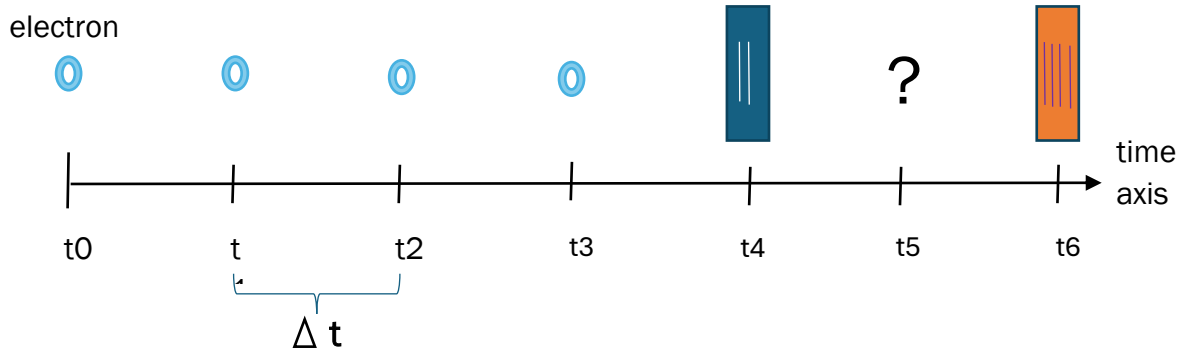
The electron is fired towards the slits → it moves towards them → it hits the slits → it somehow moves like a wave beyond the wall → hits the back panel and forms the fringes

If we think about it, we want to see this whole process happening kind of like an animation. This is where we hit our first wall (with no slits this time): the electron's motion and interactions are, in real life, carried out continuously over time. In our digital environment it is impossible to represent a perfectly continuous concept, as it is infinite.

So here we face the discretization problem. How do we represent something infinite in a finite environment such as our computer's memory? In our current situation, we need to somehow make time finite. The solution to this problem is to capture so many consecutive events that the missing time spans are simply not observable. This tactic can be compared to how videos are actually represented: a lot of ordered images played fast enough that they appear as one perfectly continuous visualisation.

In fact, we will use this exact video / film tactic when visually representing the simulation at the end. We will take snapshots of our calculations at fixed time moments and concatenate them into a movie-like animation.

The beautiful thing about this approach is that this also works perfectly well for the actual computation of the physics behind the project, not just for the final representation. The movement of the electron would normally be carried out continuously in our real time, but with this approach we can calculate all the occurring values only at certain moments! If only we had a way of predicting how our electron evolves over time...



Enter the *Partial Derivative of the Wave Function with respect to time* which we talked about last chapter. This is the crucial tool that allows us to evolve our simulated system in time! Now keep in mind that this actually tells us how the *wave function* is supposed to behave in the future, but how does this translate into us knowing how the electron behaves? Well, we know that the wave function is a representation of the electron's *quantum state* but the question persists: where does this get us after all of this?

What do we even compute?

By answering this question, everything we talked about so far should finally make sense and show why it's even relevant to us.

Recall how we said before that:

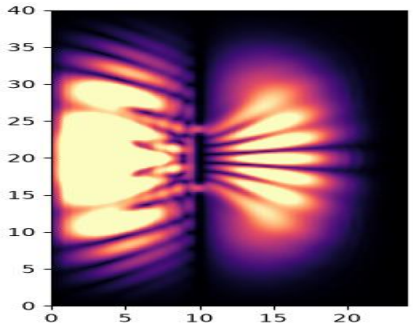
"The wave function outputs a complex number called the probability amplitude. While this complex number carries a lot of information, we are only interested in one use of it, namely computing the probability density $|\Psi^2|$. *Summing up this probability density over an area will ultimately give us the probability of finding our particle in that space at the time moment ."*

This piece of information is crucial because it reveals the actual meaning of this whole experiment and also points us in the direction we should follow going forward: The electron simply does not have a perfectly defined position in space.

This is the reason we didn't just use a simple Newtonian mechanics approach to all of this. Something like modeling the electron as a point and giving it a velocity vector, moving it forward and seeing what happens, would not give us the expected results. The electron simply does not behave in a way that such an approach could describe it. Instead, the wave function gives us through the probability density a set of probabilities of finding the electron at certain positions. When the electron is first "shot", the probability of it being located around its starting point is high, while everywhere else very low. As the electron progresses towards the slits, the probability remains high around this moving area.

The question that arises now is: what happens with these probabilities when the electron reaches the area around the slits? And this is exactly where the wave-like behaviour appears: As the high probabilities obviously appear in the slits and not in the wall, they move forward past the slits and clash like water waves, creating the interference pattern. Should the electron not behave like a wave, it would continue normally in two separate lines, hitting the back panel and creating exactly what we would all expect. As the interference pattern appears, it moves forward towards the panel and the final result consists of the fringes, which are actually just high-probability zones where the electron is likely to be found after it completes its travel, separated by low-probability zones. Now, we have never observed (and cannot observe) probabilities directly, but we can observe what happens when we put them to the test. This is exactly why we shoot multiple electrons: as they land in the zones with higher probability, they literally form the fringes by accumulating in those areas.

So to answer the question once and for all: our computational goal is to obtain a series of probabilities that represent the chances of an electron being present at certain points in space at certain time moments. More precisely, for each time moment in our discretization we need to calculate the wave function values, which in turn can give us the probabilities, while at the same time allowing us to evolve the system using the time derivative. This may sound very complex at first, but I strongly recommend continuing onward as the next chapter will take care of the actual implementation.



IV. The Digital Representation of the System

Now that we understand what our computational target is and we have an idea about what we need to simulate, we need to approach the problem of how to represent our system in the actual program.

An intuitive way that may cross many people's minds is to set our scene on a matrix. This comes from the fact that we want to simulate our experiment in a 2D plane (as we settled at the beginning). It might seem surprising that this is actually the way we will stick with until the end, no upgrades or mysterious changes here.

We do have one problem, though: we previously talked about how we need to discretize time, as our computer cannot handle infinitely many values. This same problem arises with space: we cannot represent the continuous space in the digital representation, so that is why we will also discretize it.

Overall, our space-time representation will consist of:

- A matrix representing the plane in which the events take place. We will denote the fictional measured width of each matrix cell by , and the height by . These units are only useful to us in calculations. This way we can discretize the continuous space by only referring to “regions” of it, the matrix cells.

- A hypothetical time axis, which we will advance along by utilising a simple for-loop. This for-loop will only pass through certain time moments. We will evolve the system jumping from time moment to time moment, calculating the next values from our current information, while the difference between any two such moments will be denoted by Δt and, just like in the space discretization, will only be useful to us in calculation.

Keep in mind that this approach works because we can make the deltas so small and the time moment / space regions so frequent that they are simply not visible to us and do not impact our simulation. For example, even though we do not know the precise values of the wave function between time moments t_x and t_{x+1}, if they are close enough apart and we analyse enough time moments, that “lost” time period of length Δt becomes negligible. The same applies to the space discretization: even if we do not monitor the particle's behaviour between two cells, the effect is trivial as long as the finite points we analyse are numerous enough to make the disruptions irrelevant.

Other important aspects of the simulation environment are the barriers our particle may encounter over the course of events. The representation of these aspects is not as intuitive as the discretization of time and space may have been. Here we have two main components:

- The wall with the slits
- The boundaries of the arena (margins of the matrix)

The reason we split these into two categories is that they should behave differently for us to achieve the best result in the final visualisation.

Starting with the wall, this barrier should promptly and abruptly block the electron's passing as it reaches its proximity. The way we implement this is by assigning the zones that contain the wall a high potential energy ( ) value. The empty areas of our matrix will preferably have the potential energy equal to 0, or a low value in any case. While this is not an accurate representation of reality, as we mentioned before when talking about constants in the Schrödinger Equation, the values are not relevant for the final result and we are not required to work in the S.I. (thus we purposely avoid using realistic values in order to cut the need to calculate precise amounts and keep units in line everywhere, thus also adding floating point calculation noise to our program).

The arena boundaries are different in that treating them the same as our wall will create some unexpected noise in our final image. An interesting aspect about how we represent the wall is that the electron will have a high probability of "bouncing" off it backwards. This is quite intuitive as the particle is not fired precisely through the slits and even then it is not perfectly located, thus there is a chance of hitting the wall directly and causing a rebound effect. The reason this is not desirable for the boundaries is that on the right side of the wall the electrons will hit the detection panel, creating this exact kickoff effect, which will interfere with the fringes we want to create. This can be observed as a set of vertical dark stripes in a snapshot. Thus, the way to handle this is to add an "absorption padding" to the margins, which gradually dissipates and stops the particle instead of abruptly clashing with it. This is called "Complex Absorbing Potential" (CAP).

With this we have finished defining the environment for our simulation, but one thing still remains: the actual object of all the interactions: the electron. The wave function is the tool that describes how a particle evolves through time and space, but it doesn't help us define it in our program. For this we will use one more auxiliary concept, as we need a first instance of our particle at time moment t_0 to evolve from. This initialisation of our electron will be achieved through the implementation of the “Gaussian Wave Packet”. Continue to the next chapter to see everything we talked about in action!

V. The Implementation

With this chapter we finally reach the apex of our journey: the actual implementation of the simulation. This is arguably the most complex part of this project, so make sure to take your time, understand everything before moving forward and don't rush the important parts. We will work our way from the simple parts to the more challenging ones.

1. Implementation of the Spatial Representation

Starting off with the space representation, this is, as we talked before, a simple grid. What we need specifically for this is a pair of matrices: one for actually holding the wave function values we continuously calculate and one for the potential energy values in space. Note that these matrices should both hold complex values, firstly because the wave function returns complex values, and secondly, because the *Complex Absorbing Potential* we implement to treat the margins is also complex-valued. Thus we may summarise this as:

$\Psi[\,][\,]$ = the matrix holding the wave function values (complex)

$V[\,][\,]$ = the matrix holding the complex absorbing potential values (complex)

Initialising the CAP matrix is done by the following rule:

$$V[i][j] = a - i_{\mathbb{C}} \cdot W[i][j]\,,\ a \in \mathbb{R}\,,\ a = 20\ if\ (i,j)\ wall,\ else\ a = 0$$

$$W[i][j] = \eta \cdot \left(f_x(j) + f_y(i) \right)$$

At first glance this may look quite abstract, and you would be right to question it. What this mathematical definition actually does is tell us how to assign the complex absorbing potential values to the matrix. For any positions that are not in the padding range near the boundaries, we actually store simple real numbers (for example the wall or the interior empty space), this being just the potential energy V . The padded regions get an increasing complex value W as we approach the limits of the arena, calculated separately for the x and y coordinates. Notice that in our representation the x coordinate corresponds to the j columns of the matrix, while the y coordinate corresponds to the i rows of the matrix.

Below you can see a table of the exact branches of the $f_{x,y}$ functions and the constants we use in the formulas.

j	$f_x(j)$
$j < x_b$	$\left(\frac{x_b - j}{d}\right)^n$
$x_b \leq j \leq x_r$	0
$j > x_r$	$\left(\frac{j - x_r}{d}\right)^n$

i	$f_{y(i)}$
$i < y_b$	$\left(\frac{y_b - i}{d}\right)^n$
$y_b \leq i \leq y_r$	0
$i > y_r$	$\left(\frac{i - y_r}{d}\right)^n$

$$n = 2$$

$$\eta = 6$$

d = the padding width (distance from the first column / first line / last column / last line to the end of the padding)

$i_{\mathbb{C}}$ = complex i (to differentiate from the matrix columns i)

2. Implementation of the Particle Representation

Moving on to the initialisation of our electron, let's take a closer look at the *Gaussian Wave Packet and* find out what it is and what it actually looks like:

$$\Psi[i][j] = e^{-\frac{(j-x_0)^2}{4\sigma_x^2} - \frac{(i-y_0)^2}{4\sigma_y^2}} \cdot e^{i_{\mathbb{C}}(k_x \cdot j + k_y \cdot i)}$$

We talked about how the wave function allows us to determine how the particle evolves over time, but it doesn't help us in defining a first instance of the particle at the very first time moment, the "initialisation" needs to be done in a different way. The Gaussian Wave Packet is basically a manual creation of the wave function. It is split into two parts:

- The envelope (real part): $e^{-\frac{(j-x_0)^2}{4\sigma_x^2} - \frac{(i-y_0)^2}{4\sigma_y^2}}$

  This seemingly frightening expression has the role of locating the chosen particle at a certain position in our designed space. Without this, the electron would be located practically everywhere, like a field. (x_0, y_0) represents the starting position of the electron; we will consider this the point it is shot from by the electron gun. σ_x and σ_y both define the electron's localisation: the smaller the values, the more precisely it is located around the starting point.

- The phase (complex part): $e^{i_{\mathbb{C}}(k_x \cdot j + k_y \cdot i)}$

  This shorter part is what gives the electron its starting momentum. Depending on what values we choose for k_x and k_y, the electron's probability will start drifting in the pointed direction. For example, $k_x > 0$ will direct the electron to the right, while $k_y < 0$ will make it drift down.

So in practice we will strictly use the Gaussian Wave Packet for the very first calculation of Ψ . From the time moment t_1 onward we can rely solely on the Schrödinger Equation and the wave function itself, but at the beginning we need something to define the starting position and direction for our particle.

One last important part to the Gaussian Wave Packet is the normalisation constant A . It is intuitive that the sum of probability densities of finding the electron at all points in space should be equal to 1. The normalisation constant is what scales the values at the beginning so they actually respect this. After we initialised Ψ by the Gaussian Wave Packet formula, all we have to do is calculate A as follows:

Then just multiply to every element . The great thing about this process is that the normalisation will be preserved over the whole evolution we are about to carry out, so we do not have to worry about the probabilities being precise or normalised ever again over the course of the program.

3. Evolving the System

At the beginning of this chapter I mentioned how this would be "the apex" of our journey. It is safe to say that following the topic we will discuss right now, we will have officially summited our "Everest". Recall that this whole project is about implementing a *simulation*. The actual simulation is the evolution of the system in time, so everything we discussed up until now was actually just preparing the grounds on which we can now build our house, everything from understanding the experiment, defining our problem, the environment and events that take place in it, it all led up to this moment.

As we discussed the Gaussian Wave Packet, we noted that we used this concept only to initialise our particle's starting state, as we had nothing to evolve it from. Recall that the *Schrödinger Equation* is the tool that allows us to move the system forward in time. It achieves this by offering us a relation between the wave function and its time derivative . The question is, how does this help us?

The Gaussian Wave Packet grants us the values of Ψ at time moment t_0. Using the *Schrödinger Equation,* we can obtain the time derivative of Ψ at any time moment t_k. Let's stop for a second and think, what do we actually need to compute for our simulation? Our target is continuously obtaining the value of Ψ as we go forward in time. We quickly realise that we simply lack the tool that allows us to obtain Ψ at $t1$, from Ψ at t_0! In fact, we lack the necessary relation to advance at literally any moment in time; our most crucial mechanism simply has no motor right now. But what we do know is how to obtain Ψ 's time derivative at any time moment t_k. Can this somehow help us to compute Ψ at a further time moment t_{k+1}?

Introducing The Limit Definition of a Derivative. Let's see how this "primitive" form of a derivative makes our whole simulation possible:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

The first question most people would ask when seeing this appear in our discussion is: "does this even make sense in our context?", after all, this is the definition used for simple real-valued functions, so how does it apply to our terrible complex-valued, multivariable Ψ ? The simple answer to this question is that this works perfectly fine in our context. Let's replace some variables above with things we are more familiar with:

$$\frac{\partial \Psi}{\partial t} = \lim_{\Delta t \to 0} \frac{\Psi(t + \Delta t) - \Psi(t)}{\Delta t}$$

The beautiful thing about what we just did is that even though it is a simple substitution of a relation with terms from our problem, it is just not intuitive at first. Seeing *partial derivatives* and *multivariable complex functions* can be quite frightening and might seem like it just won't *fit,* well, it does! In order to actually make use of what we just assembled above, we have to utilise two more little calculus tricks:

Firstly, a quite pretty observation about this definition of the derivative is that if h (or in our case Δt) is finite and fixed (not approaching 0), then the whole derivative can be approximated to a very simple:

$$\frac{\partial \Psi}{\partial t} \approx \frac{\Psi(t + \Delta t) - \Psi(t)}{\Delta t}$$

Also, performing a little magic on this relation, we obtain:

$$\Psi(t + \Delta t) \approx \frac{\partial \Psi}{\partial t} \Delta t + \Psi(t)$$

And with a simple but miraculous realisation, by supposing we compute this at a time moment t_k, we obtain:

$$\boxed{\Psi(t_{k+1}) \approx \frac{\partial \Psi}{\partial t} \Delta t + \Psi(t_k)}$$

With this final relation, we officially discovered the key to our whole project: a way to calculate continuously over time. You may find this approximation called *Euler's Method* (when talking about ordinary differential equations) or *Finite Difference* (referring to simply approximating a derivative).

One more thing we need to note before advancing is the computation of the derivative itself. Recall that we can achieve this by utilising the Schrödinger Equation:

$$\frac{\partial \Psi}{\partial t} = \left(\frac{ih}{2m} \nabla^2 - \frac{iV}{h} \right) \Psi$$

are all constants. We have resolved the terms involving and , but what about the Laplacian Operator ? We stated earlier that this would be taken care of cleanly and easily, so this is what we will touch on right now. We can compute the Laplacian at any position in space for our current moment , utilising only the current Ψ values:

$$\nabla_x^2 = \Psi[i][j-1] + \Psi[i][j+1] - 2\Psi[i][j]$$

$$\nabla_y^2 = \Psi[i-1][j] + \Psi[i+1][j] - 2\Psi[i][j]$$

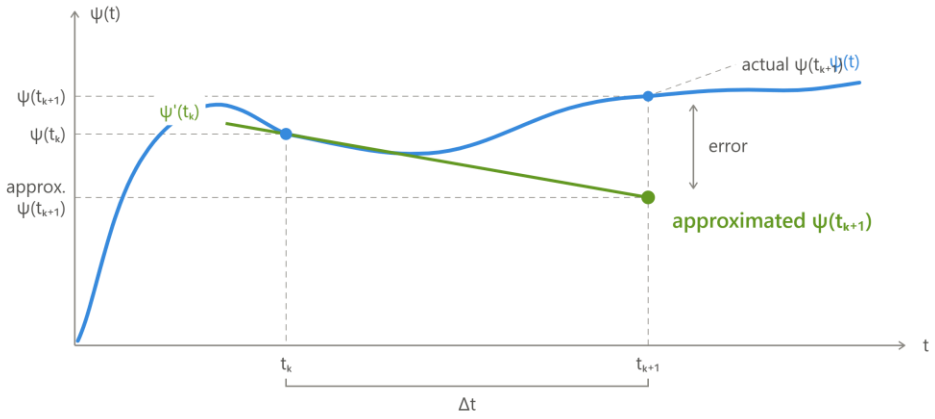
And thus $\nabla^2 = \nabla_x^2 + \nabla_y^2$. That simple!

Take a moment to appreciate what this whole process led up to, then I would encourage you to start actually coding a first version of everything we talked about. At this moment, implementing a first working version is entirely within reach, as we have gone through everything you should know before going head-first into implementing the simulation.

Going further, we will discuss an important enhancement that must be implemented to get the expected results. A project made with what we went through up until now will work, but it will be in desperate need of an upgrade: the *Runge-Kutta Method* (RK4).

4. Upgrade

Before we discuss the Runge-Kutta method, we should first understand why we even need this upgrade. Besides, we barely got our first version of the simulator going, right? The reason behind this change is that the way our program currently works sadly does not perform well enough for our final result. The point of our simulation is to obtain, in the end, the famous fringes of the double slit experiment; this was in fact our goal from the beginning. While everything we have discussed until now is perfectly correct, our problem resides in something we have encountered before: approximations and discretizations. Since the method we used above to calculate further values of Ψ is just an approximation, it will have some kind of error. The problem with our calculation is that this error scales rather rapidly, and our approximation is a rough one by the standards imposed by the goal of the experiment. In order to see why this happens we should look at an example and see how what we implemented above actually works:



. The way we currently calculate the following values of Ψ assumes that the slope at our current time moment is a good enough indicator for where the next point will be located. A situation where this assumption shatters is exactly what is depicted above: should the slope abruptly change within the discretized time interval of length Δt between our current time moment and the next, we will obliviously just pass right over it. This

problem is a direct consequence of the time discretization which we talked about earlier on in this article. This problem can in turn direct our calculation farther away from the correct values of Ψ , thus resulting in incorrect computation and, consequently, an erroneous final result. On top of this, our norm ($|\Psi^2|$) drifts away as we evolve the system using this method, meaning the total probability (which should be 1 over the entire system) grows and thus doesn't represent the correct probability.

Thinking of a solution, the first one that comes to mind is making Δt smaller. While this can work, it increases our computational workload and only postpones the instability (as the norm still diverges eventually).

Enter Runge-Kutta method (RK4). The basic idea of RK4 is to analyse the slope more frequently, inside of our time interval too, not only at its ends. The procedure:

1. Calculate $\frac{\partial \Psi}{\partial t}$ at time moment t_k
   Let this be S_1

2. Find Ψ at time $t_k + \frac{\Delta t}{2}$, based on the slope found at t_k

3. Calculate $\frac{\partial \Psi}{\partial t}$ at time $t_k + \frac{\Delta t}{2}$
   Let this be S_2

4. Find Ψ at time $t_k + \frac{\Delta t}{2}$ but based on the slope found at time $t_k + \frac{\Delta t}{2}$ ()

5. Calculate $\frac{\partial \Psi}{\partial t}$ at time $t_k + \frac{\Delta t}{2}$ again (basically enhanced, using S_2)
   Let this be S_3

6. Find Ψ at time moment t_{k+1} using slope found at time $t_k + \frac{\Delta t}{2}$ (S_3)

7. Calculate $\frac{\partial \Psi}{\partial t}$ at time moment t_{k+1}
   Let this be S_4

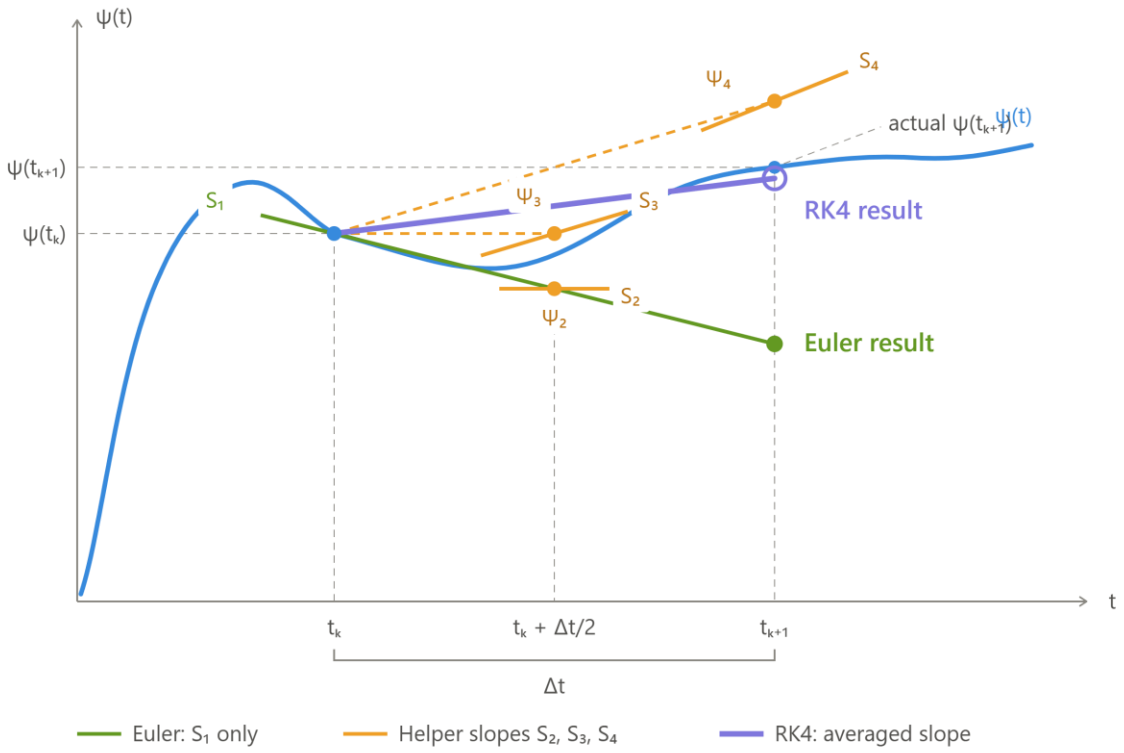
8. Finally, compute Ψ at time moment t_{k+1} with slope = $\frac{S1+2\cdot S2+2\cdot S3+S4}{6}$

$$\Psi(t_{k+1}) = \frac{S1 + 2 \cdot S2 + 2 \cdot S3 + S4}{6} \cdot \Delta t + \Psi(t_k)$$

Note:

$$t_k + \frac{\Delta t}{2} = intermediate\ time\ moment$$

$$t_k + \Delta t = t_{k+1}$$



Above you can see the same graph as the one used to display Euler's method, only now we apply RK4. The process calculates everything in this order:

S1 → Euler result , S2 → S3 → S4 → RK4 result

With this we reach the end of the implementation chapter and rapidly approach the end of our whole project. You now officially know everything you need to make a perfectly functional, correct and visually sound simulation of the double slit experiment, congratulations! Proceed to the last chapter and enjoy the end of our journey.

VI. Conclusions

The results of the implementation we just assembled, as we mentioned before, consist of a set of matrices filled with numbers, all representing probabilities of the localisation of the electron. You may be puzzled by this and ask yourself “how do I even interpret this?”. A very simple but effective way is to plot one or even all matrices as “heat maps”, or something similar. As a recommendation, Python is exceptionally good at plotting data of this form, so try out Matplotlib. Of course, any other plotting library or language will do just as well.

A highly recommended way to structure your project is to have a UI, a plotter / representation program and the central computing program (this is the part that this whole article actually described). These things are very versatile and thus completely up to you. You could allow the user to change the constants and see how the simulation behaves, you could add different scenarios with different numbers of slits, and the possibilities are endless.

Going through more than 20 pages of mathematics, physics and computer science explanations is not easy, but it is not the end either. Pick up a text editor, a helpful AI agent, this article, a pen and some paper, ask a friend or a teacher, and enjoy the process of actually creating your own simulator of a quantum phenomenon. It probably sounds just as frightening as it did before you read this article, but I can assure you it is not. At the very least you now have one thing you did not have before: the knowledge that it is actually possible and within your reach. So if you leave with one thing from this paper, it should be the idea that you can build far more than you expect with enough patience, will and research. I highly encourage you to take on more challenging projects like this one and approach them like we did before: break down unknown concepts, plan out your implementation, find your target and use help for the things you do not know.

“If I have seen further it is by standing on the shoulders of Giants.”

Isaac Newton